Lesson 1

Material

https://github.com/unixfile/learning-python-2026

Today

- Tooling setup: uv and IPython
- Interactive mode
- Variables and naming conventions
- Scripts and execution
- Comments and object inspection
- Strings and f-strings

1 Tooling setup

The uv package/project manager tool

https://docs.astral.sh/uv/getting-started/installation/

Optional uv autocompletion

https://docs.astral.sh/uv/getting-started/installation/#shell-autocompletion

Directly run command from package

```
$ uvx cowsay -t "Hello world!"
```

Run Python w/o installing

```
$ uvx python
```

Run the IPython enhanced shell

```
$ uvx ipython
```

2 Tooling setup, continued...

Installing Python, as opposed to running with uvx

```
$ uv python install --default --preview
```

Run Python when installed

```
$ python3
```

Optionally install other tools

```
$ uv tool install <package>
```

The Python Package Index

https://pypi.org

Not covered

- Legacy tooling, now part of uv: pip/pipx, venv
- Editor/IDE: Find one you like. Many use VS Code/Codium

3 Interactive mode

Invoking the IPython shell/repl

```
$ uvx ipython
```

Using normal commands in IPython, shell convenience only

```
In : !ls
```

. . . and some Python

```
In : print("Hello world!")
Out: 'Hello world!'
```

Auto-print convenience, not proper Python

```
In : "Hello world!"
Out: 'Hello world!'
```

4 Interactive mode, continued...

Variable assignment

```
In : some_number = 42
In : some_number
Out: 42  # auto-print, IPython only
In : type(some_number)
Out: int
```

5 Variable naming conventions

```
my_variable       variables, functions
MyClass           classes
MAX_SIZE          constants
```

- More on classes later

6 Special variables

```
_ = "throw-away variable, by convention"
_my_var = "this is a private variable"
__dunder__  # Special built-in names, eg __main__, __name__
```

7 Non-interactive (real) programming

Script with input/output

```
# script.py
ip = input("Input an IP address: ")
print(ip)
```

Run script

```
$ python3 script.py
```

8 Alternative execution for POSIX systems

Add shebang line on-top

```
#!/usr/bin/env python3
ip = input("Input an IP address: ")
print(ip)
```

Make executable

```
$ chmod +x script.py
```

Execute

```
$ ./script.py
```

9 Comments

```
# This is a comment
ip_addr = input("Enter an IP address: ")
print(ip_addr)

"""
This is a multiline comment.

Write your essay here.
"""
```

10 Object inspection

```
help(my_var)   # Provides object documentation

dir(my_var)    # List object members
```

- In Python, almost everything is an object

11 Python strings

```
s1 = "string with double quotes"
s2 = '...but single quotes also work'
s3 = """
    This is a
    multiline string.
    Or is it a comment?
"""
```

12 Some string methods

```
In : s = "banan melon kiwi citron"
In : s.split()
Out: ['banan', 'melon', 'kiwi', 'citron']

In : ip = "192.168.2.1"
In : parts = ip.split(".")
In : ".".join(parts)
Out: '192.168.2.1'
```

13 Chaining methods

```
In : s = "   Some string   "
In : s.lower()
Out: '   some string   '

In : s.lower().strip()
Out: 'some string'
```

14 Python f-strings

```
In : name = "Alice"
In : f"Hello, {name}!"
Out: 'Hello, Alice!'

In : a, b = 5, 10
In : f"The sum of {a} and {b} is {a + b}."
Out: 'The sum of 5 and 10 is 15.'

In : pi = 3.14159
In : f"Pi is {pi:.2f}."
Out: 'Pi is 3.14.'
```

15 More on strings

```
In : "ocean" in "Explore the ocean depths"
Out: True

In : "Hello" + ", " + "Alice" + "!"
Out: 'Hello, Alice!'

In : for c in "Hello":
   :     print(c)
H
e
l
l
o
```